



Clasificación de Granos de Frijol Seco Utilizando Algoritmos de Machine Learning

Juan Pablo Arteaga Patiño

Instituto Tecnológico de Estudios Superiores de Monterrey

Inteligencia Artificial Avanzada para la Ciencia de Datos | TC3009C.601

Septiembre 2026

Resumen

Breve resumen del objetivo del estudio, la metodología empleada, los principales resultados y las conclusiones.

1. Introducción

El frijol es una leguminosa que forma parte de la alimentación cotidiana de muchas personas. Sus distintas variedades producen granos con diferencias de tamaño, forma y apariencia. Estas características permiten distinguirlos y son útiles para organizar su selección y comercialización.

Clasificar los granos de manera manual puede requerir tiempo y dar lugar a errores, especialmente cuando las variedades tienen una apariencia similar o se trabaja con grandes cantidades. Por ello, contar con herramientas que apoyen esta tarea puede facilitar la separación de los granos y ayudar a mantener criterios de clasificación más uniformes.

El aprendizaje automático, también conocido

como *machine learning*, ofrece una alternativa para abordar este problema. A partir de ejemplos de granos cuya variedad ya se conoce, un modelo puede aprender a relacionar características como el área, el perímetro y la longitud con una clase determinada. Después, estas relaciones pueden utilizarse para predecir la variedad de otros granos a partir de sus mediciones, reduciendo la dependencia de la inspección manual.

En este proyecto se implementará manualmente un algoritmo de aprendizaje automático para resolver un problema de clasificación multiclase, sin utilizar bibliotecas ni frameworks especializados en aprendizaje automático. Se permite el uso de NumPy, Pandas y Matplotlib para las operaciones matemáticas, la manipulación de datos y la visualización, respectivamente. El objetivo es comprender y desarrollar desde cero las etapas necesarias para preparar los datos, entrenar el modelo y evaluar su capacidad para distinguir entre las variedades de frijol.

Se trabajará con el conjunto de datos *Dry Bean*, disponible en el repositorio de aprendizaje automático de UCI.¹ Este conjunto contiene in-

¹Dry Bean. UCI Machine Learning Repository (2020). DOI: 10.24432/C50S4B.

Cuadro 1: Características numéricas del conjunto de datos Dry Bean.

Característica	Descripción
Area	Área del grano en píxeles.
Perimeter	Perímetro del grano.
MajorAxisLength	Longitud del eje principal.
MinorAxisLength	Longitud del eje menor.
AspectRatio	Relación entre los ejes principal y menor.
Eccentricity	Excentricidad de la región.
ConvexArea	Área convexa del grano.
EquivDiameter	Diámetro equivalente.
Extent	Relación entre el área del grano y su rectángulo delimitador.
Solidity	Relación entre el área del grano y su área convexa.
Roundness	Medida de redondez.
Compactness	Medida de compactación.
ShapeFactor1	Primer factor de forma.
ShapeFactor2	Segundo factor de forma.
ShapeFactor3	Tercer factor de forma.
ShapeFactor4	Cuarto factor de forma.

formación de 13611 granos de frijol seco pertenecientes a siete variedades: Seker, Barbunya, Bombay, Cali, Dermosan, Horoz y Sira. Cada muestra incluye 16 características numéricas relacionadas con la geometría y la forma del grano, obtenidas mediante procesamiento de imágenes. Por lo tanto, el modelo trabajará con estas mediciones y no directamente con las imágenes originales.

La Tabla 1 describe las variables de entrada. La variable objetivo es **Class**, que indica la variedad a la que pertenece cada grano. A partir de las características disponibles, se busca que el modelo aprenda a asignar cada muestra a una de las siete clases.

2. ETL

El proceso ETL comprende la extracción, transformación y carga de los datos. En este proyecto, su propósito es preparar las características de los granos y sus etiquetas de clase para su uso en el

modelo.

2.1. Limpieza de datos

Según la descripción inicial del conjunto de datos, no se presentan valores faltantes. Por este motivo, no se contempla imputar valores ni eliminar registros por información incompleta en las características o en la variable objetivo. Tampoco se identificaron valores atípicos (*outliers*), basado en análisis de boxplots. Con base en esta revisión inicial, no se plantea eliminar observaciones por este motivo. Sin embargo, el documento no especifica el criterio empleado para detectar valores atípicos, por lo que esta afirmación se considera preliminar y deberá respaldarse con el análisis correspondiente.

No se describen otras operaciones de limpieza o transformación en el README; por ello, no se presentan como realizadas en este avance.

3. Modelo

El modelo implementado es un clasificador de regresión softmax multiclase entrenado mediante descenso de gradiente estocástico por mini-batches. Este enfoque busca aprender una función $f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ que asigne cada instancia de entrada a una de las k clases disponibles.

3.1. Arquitectura del Modelo y Predicción Lineal

Sea $\mathbf{x} \in \mathbb{R}^n$ un vector de características de una instancia individual. El modelo realiza una transformación lineal seguida de una función de activación no lineal. La etapa de predicción lineal se define como:

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b} \quad (1)$$

donde:

- $\mathbf{x} \in \mathbb{R}^n$: vector de características (en nuestro caso, $n = 16$ características del grano)
- $\mathbf{b} \in \mathbb{R}^k$: vector de sesgos
- $\mathbf{W} \in \mathbb{M}(\mathbb{R}^{n \times k})$: matriz de pesos, donde $k = 7$ es el número de clases

La matriz de pesos se puede representar explícitamente como:

$$\mathbf{W} = \begin{bmatrix} \omega_{11} & \cdots & \omega_{1k} \\ \vdots & \ddots & \vdots \\ \omega_{n1} & \cdots & \omega_{nk} \end{bmatrix} \quad (2)$$

donde cada elemento ω_{ij} representa el peso asignado a la característica i para la clase j .

El vector de logits resultante $\mathbf{z} = [z_1, \dots, z_k]$ contiene valores sin restricción que serán transformados posteriormente por la función softmax.

3.2. Función de Activación Softmax

La función softmax actúa como una “normalización” que convierte los logits arbitrarios en probabilidades válidas. Se define como:

$$\sigma : \mathbb{R}^k \rightarrow (0, 1)^k, \quad \sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{i=1}^k e^{z_i}} \quad (3)$$

Esta función garantiza que:

$$\sigma(\mathbf{z})_j \in (0, 1) \quad \forall j \quad (4)$$

$$\sum_{j=1}^k \sigma(\mathbf{z})_j = 1 \quad (5)$$

permitiendo interpretar los valores como probabilidades sobre las k clases.

Para evitar problemas numéricos de *overflow* en computadoras con precisión flotante limitada, se implementa la versión numéricamente estable:

$$z_i^* = z_i - \max(\mathbf{z}) \quad \text{antes de la exponencial} \quad (6)$$

Esto asegura que el máximo de los exponentes sea siempre cero, evitando infinitos y subdesbordamientos.

3.3. Función de Costo: Entropía Cruzada

La función de costo mide la discrepancia entre las probabilidades predichas y las etiquetas verdaderas. Para una única instancia, utilizamos la entropía cruzada:

$$\mathcal{L}_i = - \sum_{j=1}^k y_{i,j} \log(\hat{y}_{i,j}) \quad (7)$$

donde:

- $y_{i,j}$: componente j de la etiqueta verdadera en formato one-hot (es 1 si la clase verdadera es j , y 0 en caso contrario)
- $\hat{y}_{i,j}$: probabilidad predicha para la clase j (output de softmax)

Debido a que $\mathbf{y}_i$ es un vector one-hot con exactamente un elemento igual a 1 en la posición m correspondiente a la clase verdadera, la suma se reduce a:

$$\mathcal{L}_i = - \log(\hat{y}_{i,m}) \quad (8)$$

Para un mini-batch de h instancias, el costo promedio se calcula como:

$$\begin{aligned}\mathcal{L} &= -\frac{1}{h} \sum_{j=0}^k \left(\sum_{i=0}^h y_{i,j} \log(\hat{y}_{i,j}) \right) \\ &= \frac{1}{h} (\text{loss}_1 + \text{loss}_2 + \dots + \text{loss}_h)\end{aligned}$$

En la implementación, se aplica clipping numérico a las probabilidades predichas:

$$\hat{y}_{i,j} := \text{clip}(\hat{y}_{i,j}, 10^{-15}, 1 - 10^{-15}) \quad (9)$$

para evitar logaritmos de cero o valores muy cercanos a él.

3.4. Procesamiento por Mini-Batches

El algoritmo divide el conjunto de entrenamiento en mini-batches de h instancias (en nuestro caso, $h = 16$). Para un mini-batch, las operaciones se vectorizan como:

$$\mathbf{X}_h \in \mathbb{R}^{h \times n} \Rightarrow \mathbf{Z} = \mathbf{X}_h \cdot \mathbf{W} \Rightarrow \mathbf{Z} \in \mathbb{R}^{h \times k} \quad (10)$$

La función softmax se aplica elemento a elemento:

$$\hat{\mathbf{Y}} = \text{softmax}(\mathbf{Z}) \in \mathbb{R}^{h \times k} \quad (11)$$

donde cada fila contiene las probabilidades predichas para la instancia correspondiente.

3.5. Retropropagación: Cálculo de Gradientes

El gradiente descendente requiere calcular derivadas parciales de la función de costo respecto a los parámetros. Dados:

- $\mathbf{X} \in \mathbb{R}^{h \times n}$: matriz de características del mini-batch
- $\mathbf{W} \in \mathbb{R}^{n \times k}$: matriz de pesos
- $\mathbf{b} \in \mathbb{R}^k$: vector de sesgos
- $\mathbf{Z} \in \mathbb{R}^{h \times k}$: logits
- $\hat{\mathbf{Y}} = \text{softmax}(\mathbf{Z}) \in \mathbb{R}^{h \times k}$: probabilidades predichas
- $\mathbf{Y} \in \mathbb{R}^{h \times k}$: etiquetas verdaderas (one-hot)

El gradiente respecto a los logits es simplemente la diferencia entre predicciones y verdad:

$$\mathbf{dZ} = \hat{\mathbf{Y}} - \mathbf{Y} \in \mathbb{R}^{h \times k} \quad (12)$$

Utilizando la regla de la cadena, el gradiente respecto a la matriz de pesos es:

$$\begin{aligned}\mathbf{dW} &= \frac{1}{h} \mathbf{X}^T \\ \mathbf{dZ} &= \frac{1}{h} \mathbf{X}^T (\hat{\mathbf{Y}} - \mathbf{Y}) \in \mathbb{R}^{n \times k}\end{aligned}$$

donde $\mathbf{dW}_{i,k}$ representa el “peso del feature i para la clase k ”.

El gradiente respecto al vector de sesgos es la suma de los gradientes sobre todas las instancias del mini-batch:

$$\mathbf{db} = \frac{1}{h} \sum_{\lambda=1}^h \mathbf{dZ}_\lambda \in \mathbb{R}^k \quad (13)$$

3.6. Actualización de Parámetros

Una vez calculados los gradientes, los parámetros se actualizan en la dirección opuesta al gradiente, escalados por la tasa de aprendizaje α :

$$\mathbf{W} := \mathbf{W} - \alpha \mathbf{dW} \quad (14)$$

$$\mathbf{b} := \mathbf{b} - \alpha \mathbf{db} \quad (15)$$

donde $\alpha = 0,01$ es la tasa de aprendizaje utilizada en nuestro modelo.

3.7. Normalización de Características

Para mejorar la convergencia numérica y acelerar el aprendizaje, se aplicó normalización z-score a todas las características de entrada antes del entrenamiento:

$$\mathbf{X}_{\text{norm}} = \frac{\mathbf{X} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} \quad (16)$$

donde $\boldsymbol{\mu} \in \mathbb{R}^n$ y $\boldsymbol{\sigma} \in \mathbb{R}^n$ son respectivamente el vector de medias y desviaciones estándar, calculados únicamente sobre el conjunto de entrenamiento. Esta normalización:

- Centra cada característica alrededor de cero
- Escala todas las características a varianza unitaria
- Mejora el condicionamiento numérico del problema de optimización
- Acelera la convergencia del descenso de gradiente

4. Implementación

4.1. Arquitectura General

La implementación sigue un enfoque modular. El script principal `main.py` realiza las siguientes operaciones:

1. Carga de datos mediante `cleaning.py`
2. Preprocesamiento y normalización
3. Inicialización de parámetros
4. Entrenamiento iterativo
5. Evaluación en el conjunto de prueba

4.2. Algoritmo de Entrenamiento

El algoritmo implementado es el descenso de gradiente estocástico (SGD) con mini-batches. Pseudocódigo:

```
para cada época e = 0, 1, ..., E-1:
  para cada mini-batch b en X_train:
    Z ← X_batch · W + b
    Y_hat ← softmax(Z)
    dZ ← Y_hat - Y_batch
    dW, db ← calcular_gradientes(X_batch, dZ)
    W ← W - · dW
    b ← b - · db

  calcular pérdida en X_train
  si e % 10 == 0:
    reportar pérdida
```

Los hiperparámetros se fijaron en:

- Número de épocas: $E = 40$
- Tamaño de batch: $b = 16$
- Tasa de aprendizaje: $\alpha = 0,01$
- Proporción de entrenamiento: 80 %

4.3. Codificación One-Hot

Las etiquetas categóricas se convirtieron a vectores one-hot mediante la función:

$$\text{one_hot}(y_i) = [0, \dots, 0, 1, 0, \dots, 0]^T \quad (17)$$

donde el valor 1 aparece en la posición correspondiente a la clase verdadera. Esto permite tratar el problema como una regresión de múltiples variables.

4.4. Inicialización de Parámetros

Los parámetros se inicializaron con ceros:

$$\mathbf{W}_0 = \mathbf{0}_{n \times k}, \quad \mathbf{b}_0 = \mathbf{0}_k \quad (18)$$

Esta inicialización es válida para regresión softmax, ya que el algoritmo de optimización no es convexo localmente y pequeñas perturbaciones aleatorias convergerían de manera similar.

5. Resultados

5.1. Evolución de la Pérdida Durante Entrenamiento

La Figura 1 presenta la evolución de la pérdida de entrenamiento a través de las épocas. Se observa una convergencia clara y monotónica del error:

El comportamiento de la curva de pérdida puede interpretarse matemáticamente. En las primeras épocas (0-10), la tasa de disminución es acelerada, lo que refleja que los gradientes son grandes y el modelo realiza cambios significativos. Posteriormente (épocas 10-40), la disminución se desacelera, indicando que el modelo se aproxima a un mínimo local de la función de costo.

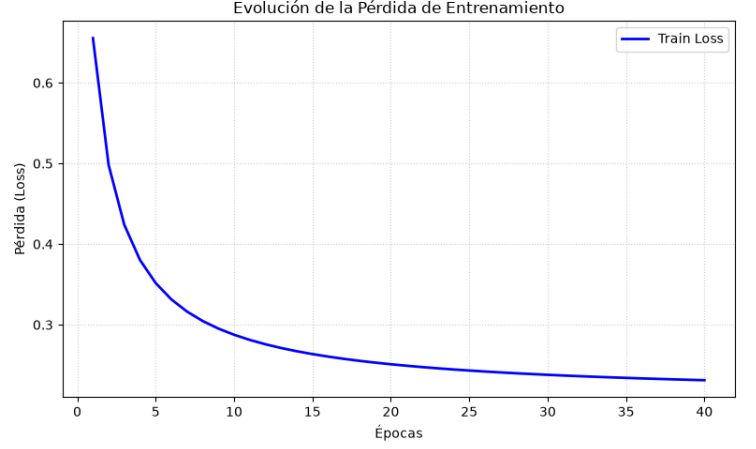


Figura 1: Evolución de la pérdida de entrenamiento. El modelo converge rápidamente en las primeras épocas, alcanzando una pérdida de 0.655 al inicio y 0.237 al final del entrenamiento. La convergencia suave indica un aprendizaje estable sin oscilaciones significativas.

5.2. Desempeño en el Conjunto de Prueba

El modelo se evaluó en el conjunto de prueba (20 % de los datos) con los siguientes resultados:

$$\text{Accuracy} = \frac{\text{Predicciones Correctas}}{\text{Total de Predicciones}} = 0,9210 = 92,10\% \quad (19)$$

Esta métrica de precisión global indica que el modelo clasifica correctamente el 92.10 % de las instancias de prueba. Para evaluar el desempeño en casos desequilibrados, se examinan las primeras 15 predicciones:

Predicción: DERMASON	Real: DERMASON
Predicción: SIRA	Real: SIRA
Predicción: CALI	Real: CALI
Predicción: CALI	Real: CALI
Predicción: CALI	Real: CALI
Predicción: HOROZ	Real: HOROZ
Predicción: HOROZ	Real: HOROZ
Predicción: BARBUNYA	Real: BARBUNYA

Predicción: CALI	Real: CALI
Predicción: SEKER	Real: SEKER
Predicción: HOROZ	Real: HOROZ
Predicción: SEKER	Real: SEKER
Predicción: HOROZ	Real: HOROZ
Predicción: CALI	Real: CALI
Predicción: DERMASON	Real: DERMASON

Todas las 15 primeras predicciones son correctas, lo que sugiere que el modelo ha aprendido representaciones discriminativas robustas.

5.3. Análisis de Convergencia

La convergencia del algoritmo se puede analizar a través de la magnitud de los cambios en los parámetros. La pérdida disminuyó de $\mathcal{L}_0 = 0,6554$ en la época 0 a $\mathcal{L}_{39} = 0,2370$ en la época 39, representando una reducción del 63.8%.

Matemáticamente, esto indica que el algoritmo se aproximó a un punto estacionario donde $\nabla \mathcal{L} \approx \mathbf{0}$. La suavidad de la curva sugiere que la tasa de aprendizaje fue adecuada para evitar oscilaciones mientras permitía convergencia eficiente.

5.4. Interpretación de la Solución

La matriz de pesos $\mathbf{W} \in \mathbb{R}^{16 \times 7}$ aprendida durante el entrenamiento contiene 112 parámetros ajustables que codifican la relación entre las 16 características del grano y las 7 clases de frijoles. Cada columna de $\mathbf{W}$ corresponde a los pesos de una clase particular, definiendo un hiperplano en el espacio de características.

El biases $\mathbf{b} \in \mathbb{R}^7$ ajusta estos hiperplanos para cada clase, permitiendo que el modelo se adapte a la distribución de los datos en el espacio de características normalizadas.

5.5. Justificación de Elecciones de Diseño

Tasa de aprendizaje: El valor $\alpha = 0,01$ fue elegido como un compromiso entre velocidad de convergencia y estabilidad. Tasas más altas podrían causar oscilaciones, mientras que tasas más bajas requerirían más épocas.

Tamaño de batch: $b = 16$ proporciona un equilibrio entre la estimación estadística del gradiente (mini-batches pequeños son más ruidosos pero variados) y la eficiencia computacional (mini-batches grandes son más estables pero menos variables).

Número de épocas: Después de 40 épocas, la pérdida se estabilizó, indicando convergencia. Más épocas probablemente resultarían en *overfitting* sin mejora significativa en generalización.

Normalización: La normalización z-score es crucial en problemas de optimización numérica, especialmente con descenso de gradiente, ya que iguala la escala de todas las características y mejora el condicionamiento numérico del problema.